

Revisao Arquitetural das Entidades Persistidas

Data de referencia: 2026-04-05

Base analisada:

- 'eTasks-server.Models/Entities'
- 'eTasks-server.Core/Data/AppDbContext.cs'

Escopo

Este documento ignora DTOs e foca apenas nas entidades relacionadas ao banco de dados atualmente mapeadas pelo 'AppDbContext'.

Inventario Atual

Entidades mapeadas:

1. 'eTasksVersion'
2. 'User'
3. 'RefreshToken'
4. 'PasswordResetCode'
5. 'LoginLog'
6. 'UserSettings'
7. 'UserBonusPoint'
8. 'BonusAchievement'
9. 'UserAchievement'

Visao de Dominio

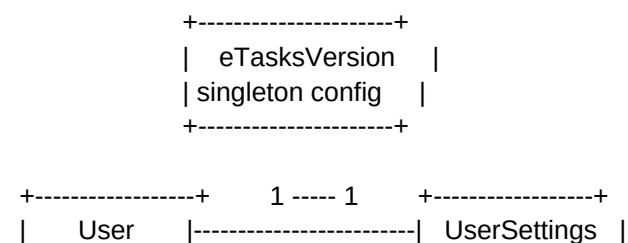
O modelo atual pode ser lido em quatro blocos:

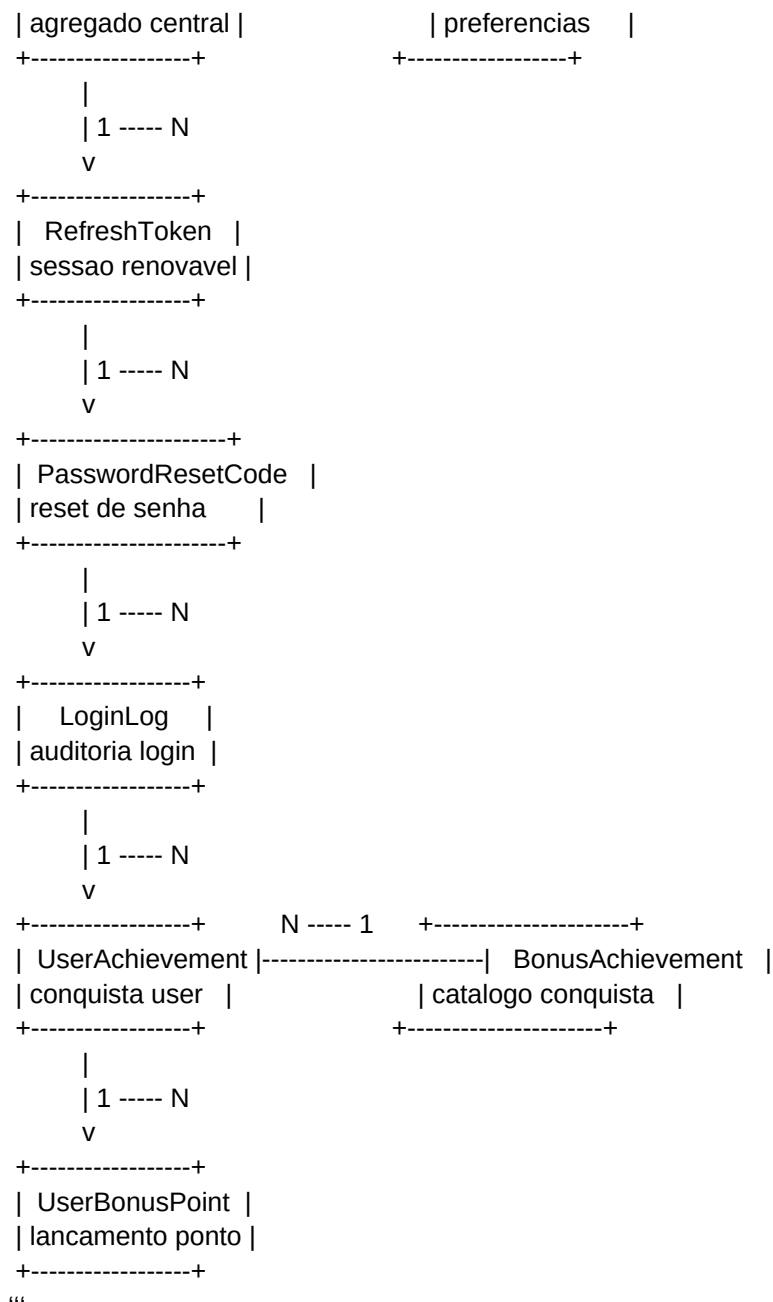
1. 'Configuracao Global'
'eTasksVersion'
2. 'Identidade e Conta'
'User'
3. 'Autenticacao e Auditoria'
'RefreshToken', 'PasswordResetCode', 'LoginLog'
4. 'Perfil e Gamificacao'
'UserSettings', 'UserBonusPoint', 'BonusAchievement', 'UserAchievement'

Essa divisao ja e suficiente para orientar futuras separacoes modulares, mesmo que hoje tudo ainda esteja concentrado sob o mesmo contexto.

Diagrama de Relacoes

“text





Observacao:

- o diagrama acima representa dependencias de dominio e mapeamentos atuais, nao necessariamente limites ideais de modulo

Analise por Entidade

'eTasksVersion'

Papel atual:

- registro singleton com dados da versao distribuida para clientes

Leitura arquitetural:

- nao se comporta como entidade de negocio rica
- funciona mais como configuracao persistida
- o fato de ter 'Id = 1' reforca que o modelo e singleton, nao catalogo

Risco estrutural:

- baixo

Oportunidade futura:

- migrar para um modulo de 'ApplicationSettings' ou 'ReleaseManagement' se o dominio de publicacao crescer

'User'

Papel atual:

- agregado central do sistema

Concentra hoje:

- identidade
- credencial
- autorizacao
- estado da conta
- metadados de ciclo de vida
- relacoes com configuracoes, tokens e gamificacao

Forca do modelo:

- simples de entender
- reduz dispersao para um sistema ainda pequeno ou medio

Risco estrutural:

- alto acoplamento
- tendencia a crescimento do agregado com regras heterogeneas

Sinais de que pode crescer demais:

- auth, admin, perfil e bonus dependem todos de 'User'
- qualquer mudanca transversal tende a tocar o mesmo agregado

'RefreshToken'

Papel atual:

- persistencia de sessao renovavel para JWT

Pontos positivos:

- modela explicitamente revogacao
- guarda 'UserAgent', o que ajuda no controle por cliente
- e um bom artefato de seguranca operacional

Risco estrutural:

- baixo

Observacao:

- compoe bem um subdominio de autenticacao

'PasswordResetCode'

Papel atual:

- token de recuperacao com expiracao e uso unico

Pontos positivos:

- estado simples
- finalidade clara
- bom encaixe transacional

Risco estrutural:

- baixo

Observacao:

- junto com 'RefreshToken', ja sugere um modulo de credenciais/sessao

'LoginLog'

Papel atual:

- trilha de auditoria para login e bloqueio

Pontos positivos:

- permite rastreabilidade de falhas e acessos
- aceita tentativas sem usuario identificado

Atencao arquitetural:

- a entidade possui relacao opcional com 'User'
- essa relacao nao esta explicitamente configurada no 'OnModelCreating'

Risco estrutural:

- medio

Motivo:

- em geral, auditoria tende a crescer rapido em volume e consulta
- vale decidir cedo se esse historico continuara no mesmo contexto transacional ou se no futuro deve ir para armazenamento mais especializado

'UserSettings'

Papel atual:

- preferencias 1:1 do usuario

Pontos positivos:

- separa configuracoes do corpo principal de 'User'
- reduz poluicao do agregado central

Risco estrutural:

- baixo

Observacao:

- esse tipo de separacao e um bom padrao para continuar seguindo

'UserBonusPoint'

Papel atual:

- ledger de pontos do usuario

Pontos positivos:

- historico preservado
- saldo derivado, nao sobrescrito
- bom para auditoria

Risco estrutural:

- medio

Motivo:

- se o volume crescer, somas frequentes podem pressionar consultas
- talvez surja necessidade de snapshot/agregado de saldo

'BonusAchievement'

Papel atual:

- catalogo mestre de conquistas

Pontos positivos:

- codigo unico
- separacao clara entre definicao de conquista e conquista adquirida

Risco estrutural:

- baixo

'UserAchievement'

Papel atual:

- associacao historica entre usuario e conquista

Pontos positivos:

- indice unico por usuario + conquista
- preserva 'PointsAtAchievement' como snapshot historico

Risco estrutural:

- baixo a medio

Motivo:

- cresce conforme a gamificacao cresce
- ainda assim, o modelo atual esta correto e previsivel

Analise de Acoplamento

Centro gravitacional

Hoje 'User' e o centro gravitacional do banco.

Impactos disso:

- regras de autenticacao dependem de 'User'
- regras administrativas dependem de 'User'
- regras de perfil dependem de 'User'
- regras de bonus dependem de 'User'

Isso nao e necessariamente um erro, mas indica um desenho muito centrado em identidade.

Coesao dos blocos

Blocos com boa coesao interna:

- 'RefreshToken' + 'PasswordResetCode' + 'LoginLog'
- 'UserBonusPoint' + 'BonusAchievement' + 'UserAchievement'

Blocos ainda muito fundidos no agregado central:

- estado da conta
- perfil base
- privilegios administrativos

Sugestoes de Modularizacao

Estas sugestoes nao implicam refactor imediato. Sao opcoes de evolucao.

Opcao 1. Modularizacao logica sem quebrar banco agora

Objetivo:

- manter o schema atual
- separar melhor responsabilidades no código

Direcao:

- modulo 'Identity'
- modulo 'Auth'
- modulo 'UserProfile'
- modulo 'Bonus'
- modulo 'AppConfig'

Vantagens:

- baixo risco
- menor custo imediato
- prepara o terreno para mudancas maiores

Quando faz sentido:

- se o projeto ainda esta evoluindo rapido e voce nao quer assumir migracoes estruturais agora

Opcao 2. Transformar 'User' em agregado mais enxuto

Objetivo:

- reduzir o peso conceitual de 'User'

Direcao:

- 'User' fica mais focado em identidade e estado da conta
- preferencias continuam em 'UserSettings'
- auth operacional fica conceitualmente em modulo proprio
- bonus fica tratado como subdominio independente referenciando 'UserId'

Vantagens:

- melhora legibilidade
- reduz pressao para jogar toda regra em 'User'

Risco:

- exige disciplina de camada, mesmo sem alterar o schema

Opcao 3. Separar contexto de auditoria

Objetivo:

- desacoplar historico operacional de login do nucleo transacional

Direcao:

- tratar 'LoginLog' como contexto de auditoria
- futuramente mover para armazenamento separado, se volume ou observabilidade exigirem

Vantagens:

- melhor escalabilidade de historico
- melhor clareza entre dado operacional e dado de negocio

Risco:

- aumenta complexidade operacional cedo demais se o sistema ainda for pequeno

Opcao 4. Evoluir bonus para subdominio proprio

Objetivo:

- preparar crescimento de gamificacao sem poluir identidade

Direcao:

- manter referencia por 'UserId'
- encapsular regras de pontos e conquista em modulo dedicado
- decidir depois se precisa saldo materializado

Vantagens:

- bonus ja esta relativamente bem separado no modelo
- boa chance de evoluir sem romper o resto do sistema

Recomendacao Pragmatica

Se o objetivo for tomar decisoes estruturais sem refactor excessivo agora, a melhor sequencia parece ser:

1. modularizar logicamente em codigo antes de modularizar fisicamente o banco
2. manter 'User' como agregado central por enquanto, mas impedir que novas responsabilidades caiam nele por padrao
3. tratar 'Auth' e 'Bonus' como dois subdominios candidatos naturais a separacao
4. considerar 'eTasksVersion' como configuracao singleton, nao como entidade de negocio central
5. explicitar no mapeamento EF a relacao de 'LoginLog' com 'User' para reduzir ambiguidade

Decisoes Estruturais que Valem Discussao

Perguntas que este modelo atual sugere:

1. 'User' deve continuar sendo o unico centro do dominio ou o projeto ja chegou no ponto de separar identidade de perfil?
2. 'LoginLog' vai continuar servindo apenas apoio operacional ou deve evoluir para trilha de auditoria mais robusta?
3. o modulo de bonus e parte central do produto ou apenas um recurso acessorio?
4. 'eTasksVersion' pertence ao dominio principal ou a um modulo de configuracao do sistema?
5. existe expectativa de crescimento em volume para tokens, logs ou pontos que justifique

pensar em estrategia de persistencia separada?

Resumo Executivo

O modelo atual esta coerente, simples e funcional. O principal risco nao esta em entidades mal desenhadas individualmente, mas no fato de 'User' concentrar demais o dominio.

Se eu tivesse que resumir a recomendacao em uma linha:

> O proximo passo mais valioso nao e quebrar o banco imediatamente, e sim separar melhor os subdominios em torno de 'User', especialmente 'Auth' e 'Bonus'.

Arquivos de Referencia

- 'eTasks-server.Models/Entities/Version/eTasksVersion.cs'
- 'eTasks-server.Models/Entities/Users/User.cs'
- 'eTasks-server.Models/Entities/Users/RefreshToken.cs'
- 'eTasks-server.Models/Entities/Users/PasswordResetCode.cs'
- 'eTasks-server.Models/Entities/Users/LoginLog.cs'
- 'eTasks-server.Models/Entities/Users/UserSettings.cs'
- 'eTasks-server.Models/Entities/Users/UserBonusPoint.cs'
- 'eTasks-server.Models/Entities/Users/BonusAchievement.cs'
- 'eTasks-server.Models/Entities/Users/UserAchievement.cs'
- 'eTasks-server.Core/Data/AppDbContext.cs'